BÁO CÁO BÀI THỰC HÀNH KỸ NGHỆ PHẦN MỀM

QUẢN LÝ PHỤ THUỘC, KHÓA PHIÊN BẢN VÀ ROBOT ĐIỀU PHỐI TỰ ĐỘNG

PACKAGE DEPENDENCY MANAGEMENT & AUTOMATED SUPPLY-CHAIN DEFENSE

- **Học phần:** Mã nguồn mở
- **Sinh viên thực hiện:** Phạm Minh Khoa-102230247
- **Kho lưu trữ GitHub (Repository):** https://github.com/Khoa35734/CatUIA
- **Nhánh làm việc chính:** `main`
- **Ngày thực hiện:** 26/10/2026
- **Trạng thái tổng thể:** HOÀN THÀNH (Triển khai, Đóng gói, Cấu hình CI/CD & Dependabot, Kiểm thử và Biên dịch PDF)

1. MỤC TIÊU THỰC HÀNH

Bài thực hành nhằm trang bị kiến thức chuyên sâu và kỹ năng thực chiến về quản trị chuỗi cung ứng phần mềm (Software Supply Chain), cụ thể:

1. **Làm chủ tệp định danh Manifest (`package.json`)**: Hiểu rõ cấu trúc siêu dữ liệu của gói phần mềm, phân loại dependencies, và cơ chế định danh dự án chuẩn hóa.
2. **Hiểu sâu quy tắc Semantic Versioning (SemVer) và toán tử Caret (`^`)**: Nắm vững ý nghĩa của `MAJOR.MINOR.PATCH`, cơ chế phân giải khoảng phiên bản (Version Ranges) và các rủi ro tiềm ẩn khi cập nhật ngoài kiểm soát.
3. **Bản chất của Lockfile (`package-lock.json`)**: Khảo sát cơ chế đóng băng đồ thị phụ thuộc (Dependency Graph), vai trò của trường `resolved` (nguồn gói) và `integrity` (mã băm mật mã SHA-512) nhằm ngăn chặn tấn công giả mạo gói hoặc sai lệch môi trường; đối chiếu lệnh cài đặt thông thường `npm install` với lệnh cài đặt tái lập tuyệt đối `npm ci` (Clean Install).
4. **Quy trình đóng gói Artifact chuẩn mực (`npm pack`)**: Xây dựng thư viện toán học mẫu `dut-core-math-utils`, tinh lọc cấu trúc tarball `.tgz` sạch sẽ thông qua trường `files` và kiểm tra toàn vẹn nội dung bằng công cụ `tar`.
5. **Giả lập quy trình phát hành an toàn (`npm publish --dry-run`)**: Kiểm tra các cảnh báo và xác thực cấu hình registry mà không làm ô nhiễm kho công khai npm Registry.
6. **Tự động hóa bảo vệ chuỗi cung ứng bằng Dependabot (`dependabot.yml`)**: Thiết lập robot giám sát định kỳ hệ sinh thái `npm` và `github-actions`, tự động phát hiện mã độc/lỗ hổng bảo mật đã công bố và sinh Pull Request vá lỗi có gắn nhãn `security-patch`.
7. **Quản trị phiên bản mã nguồn với Git & GitHub Actions**: Thiết lập luồng kiểm thử liên tục (CI) tự động xác minh tính toàn vẹn của mã nguồn trên kho lưu trữ đám mây.

2. MÔI TRƯỜNG VÀ CÔNG CỤ SỬ DỤNG

Tất cả các bước thực hành được thực hiện trên môi trường máy chủ cục bộ với các thông số phần mềm thực tế được thu thập qua lệnh kiểm tra hệ thống:

Tên công cụ / Thành phần	Phiên bản thực tế	Đường dẫn nhị phân / Ghi chú
Hệ điều hành (OS)	Windows 11 Enterprise (x64)	Khung thực thi PowerShell 5.1 & CMD
Node.js Runtime	`v24.19.0`	`C:\Program Files\nodejs\node.exe`
npm CLI	`11.17.0`	`C:\Program Files\nodejs\npm.cmd`
Git Version Control	`2.55.0.windows.3`	`C:\Program Files\Git\cmd\git.exe`
Python Engine (PDF Script)	`3.14.7`	`C:\Users\ASUS-PRO\AppData\Local\Python\bin\python.exe`
Chromium PDF Renderer	Microsoft Edge 14x	`C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe` (`--headless=new`)
Repository Remote URL	`origin`	`https://github.com/Khoa35734/CatUIA.git`

3. CẤU TRÚC THƯ MỤC DỰ ÁN

Cấu trúc cây thư mục của dự án sau khi hoàn thành toàn bộ các hạng mục:

```
D:\AIU\
├── .github/
│   ├── dependabot.yml          # Bản thiết kế cấu hình robot tự động hóa Dependabot
│   └── workflows/
│       └── ci.yml              # Pipeline GitHub Actions kiểm thử tự động trên ubuntu-latest
├── .gitignore                  # Cấu hình lọc bỏ node_modules, tarball .tgz, file rác
├── generate_pdf.py             # Script Python chuyển đổi Markdown sang HTML và biên dịch PDF chuẩn A4
├── index.js                    # Điểm vào chính (Entry point), chứa logic hàm add(a, b)
├── Lab_Package_Dependency.docx # Tài liệu đề bài gốc của bài thực hành
├── LAB_REPORT.md               # Báo cáo thực hành chi tiết bằng Markdown
├── LAB_REPORT.pdf              # Báo cáo thực hành đã được biên dịch sang định dạng PDF (A4)
├── LICENSE                     # Giấy phép phần mềm mã nguồn mở MIT
├── package-lock.json           # Tệp khóa đóng băng đồ thị phụ thuộc và mã băm SHA-512
├── package.json                # Manifest định danh siêu dữ liệu và khai báo dependencies
├── README.md                   # Hướng dẫn chi tiết dự án, cài đặt, kiểm thử và tài liệu hóa
├── Screenshot 2026-10-26 154356.png # Minh chứng 1: Giao diện Dependency Graph, 2 luồng quét & 3 PRs
├── Screenshot 2026-10-26 154434.png # Minh chứng 2: Chi tiết Job cập nhật Version update tác động PR #3
├── Screenshot 2026-10-26 154510.png # Minh chứng 3: Trạng thái Update check processed hoàn tất
└── test.js                     # Bộ kiểm thử đơn vị (Unit Tests) 7 ca kiểm thử độc lập
```

4. NỘI DUNG VÀ QUY TRÌNH THỰC HIỆN CHI TIẾT

4.1. Phần 1 — Manifest, SemVer và Lockfile

4.1.1. Khởi tạo Manifest (`package.json`)

Dự án được khởi tạo thông qua lệnh:

```
npm init -y
```

Sau đó, tệp `package.json` được tinh chỉnh cấu hình theo tiêu chuẩn thư viện chuyên nghiệp:

- `name`: `"dut-core-math-utils"` (tên viết thường, không dấu, phân cách bằng dấu gạch ngang).
- `version`: `"1.0.0"` tuân thủ chuẩn SemVer cho bản phát hành chính thức đầu tiên.
- `main`: `"index.js"` xác định tệp thực thi gốc khi được các gói khác import.
- `license`: `"MIT"` đảm bảo tính tự do và pháp lý của phần mềm mã nguồn mở.
- `files`: `["index.js"]` chỉ thị rõ danh sách các tệp phân phối được phép đóng gói vào artifact.

4.1.2. Cài đặt thư viện thử nghiệm `lodash@4.17.15`

Để tái hiện kịch bản khóa phụ thuộc và rủi ro bảo mật chuỗi cung ứng, thư viện tiện ích `lodash` được cài đặt tại phiên bản `4.17.15`:

```
npm install lodash@4.17.15
```

Kết quả ghi nhận từ terminal:

```
added 1 package, and audited 2 packages in 4s
1 high severity vulnerability
To address all issues, run:
  npm audit fix --force
```

4.1.3. Phân tích SemVer và toán tử Caret (`^`)

Tại tệp `package.json`, mục phụ thuộc xuất hiện khai báo:

```
"dependencies": {
  "lodash": "^4.17.15"
}
```

- **Semantic Versioning (SemVer)**: Định dạng theo quy ước `MAJOR.MINOR.PATCH`:
- `MAJOR` (4): Thay đổi phá vỡ tương thích ngược (Breaking Changes).
- `MINOR` (17): Bổ sung tính năng mới nhưng vẫn tương thích ngược.
- `PATCH` (15): Sửa chữa lỗi nhỏ, vá bảo mật và tương thích ngược hoàn toàn.
- **Toán tử Caret (`^`)**: Cho phép npm tự động cập nhật lên các bản vá `PATCH` hoặc phiên bản tính năng `MINOR` cao nhất có sẵn trên npm Registry, miễn là không nhảy sang `MAJOR` mới (tức là `< 5.0.0.0`).
- **Rủi ro tiềm ẩn**: Nếu không có cơ chế khóa phiên bản, hai lập trình viên chạy lệnh `npm install` ở hai thời điểm khác nhau có thể nhận hai phiên bản thư viện khác nhau (ví dụ: người A tải 4.17.15, người B tải 4.17.21). Điều này dẫn đến hiện tượng "Works on my machine" và có thể vô tình kéo theo mã độc phát tán trong bản phát hành mới.

4.1.4. Bản chất và cấu trúc của Lockfile (`package-lock.json`)

Khi chạy `npm install`, npm tự động sinh tệp `package-lock.json` đóng vai trò là "ảnh chụp tức thời" (Snapshot) của toàn bộ đồ thị phụ thuộc:

```
"node_modules/lodash": {
  "version": "4.17.15",
  "resolved": "https://registry.npmjs.org/lodash/-/lodash-4.17.15.tgz",
  "integrity": "sha512-
8xOcRHvCjnocdS5cpwXQXVzmmh5e5+saE2QGoeQmbKmRS6J3VQppPOIt0MnmE+4xlZoumy0GPG0D0MVIQbNA1A==",
  "license": "MIT"
}
```

- **Trường** `version`: Khóa cứng phiên bản chính xác đang hoạt động (`4.17.15`), vô hiệu hóa tính co giãn của toán tử `^` khi tái lập môi trường.
- **Trường** `resolved`: Ghi rõ URL máy chủ đích chính xác mà npm đã tải gói nén về, ngăn chặn tấn công đổi hướng DNS hoặc giả lập gói.
- **Trường** `integrity`: Chứa chuỗi mã băm mật mã chuẩn SHA-512 (`sha512-...`). Khi cài đặt lại, npm tính toán lại mã băm của tệp tải về và đối chiếu với giá trị này. Nếu sai lệch dù chỉ 1 bit, npm lập tức dừng quá trình và báo lỗi giả mạo gói.

4.1.5. Kiểm chứng tính tái lập bằng `npm ci` (Clean Install)

Để kiểm tra tính đồng nhất của môi trường khi làm việc nhóm hoặc chạy trên CI Server, lệnh sau được thực thi:

```
npm ci
```

Kết quả ghi nhận:

```
added 1 package, and audited 2 packages in 2s
1 high severity vulnerability
```

`npm ci` xóa bỏ hoàn toàn thư mục `node_modules` cũ và tiến hành giải nén trực tiếp dựa trên tệp `package-lock.json` mà không thực hiện phân giải lại phiên bản trên registry, đảm bảo tốc độ vượt trội (2 giây) và độ chính xác 100%.

4.1.6. Khảo sát lỗ hổng bảo mật chuỗi cung ứng bằng `npm audit`

Chạy lệnh phân tích rủi ro bảo mật:

```
npm audit
```

Kết quả phát hiện các lỗ hổng nghiêm trọng đã được công bố trên cơ sở dữ liệu GitHub Advisory Database:

- **GHSA-35jh-r3h4-6jhm**: Lỗ hổng Command Injection trong lodash.
- **GHSA-p6mc-m468-83gw**: Lỗ hổng Prototype Pollution qua các hàm can thiệp đối tượng.
- **GHSA-29mw-wpgm-hmr9**: Lỗ hổng Regular Expression Denial of Service (ReDoS).
- **Đánh giá**: Phát hiện này chứng minh một cách thực tế tầm quan trọng sống còn của việc sử dụng công cụ vá lỗi tự động như Dependabot để giám sát các phiên bản cũ đã bị khóa.

4.2. Phần 2 — Đóng gói Artifact và giả lập phát hành (`npm pack` & `npm publish --dry-run`)

4.2.1. Triển khai logic thư viện (`index.js`)

Thư viện cung cấp hàm toán học cốt lõi `add(a, b)` tuân thủ module hệ thống CommonJS:

```
/**
 * dut-core-math-utils
 * Thư viện tối ưu hóa các phép toán đại số cho sinh viên kỹ nghệ phần mềm.
 */

function add(a, b) {
  return a + b;
}

module.exports = { add };
```

4.2.2. Kiểm thử đơn vị (Unit Testing) với `test.js`

Xây dựng kịch bản kiểm thử toàn diện với 7 ca kiểm thử bao phủ các trường hợp biên:

1. Cộng hai số nguyên dương: `add(2, 3) = 5`.
2. Cộng hai số nguyên âm: `add(-5, -7) = -12`.
3. Cộng số dương và số âm: `add(10, -4) = 6`.
4. Cộng với phần tử trung hòa 0: `add(0, 42) = 42`.
5. Cộng hai số 0: `add(0, 0) = 0`.
6. Cộng số nguyên kích thước lớn: `add(1000000, 2000000) = 3000000`.
7. Cộng số thực có xử lý sai số dấu phẩy động: `add(0.1, 0.2) ≈ 0.3`.

Chạy kiểm thử:

```
npm test
```

Kết quả:

```
> dut-core-math-utils@1.0.0 test
> node test.js

--- STARTING UNIT TESTS FOR dut-core-math-utils ---
[PASS] Cộng hai số nguyên dương: add(2, 3) => 5
[PASS] Cộng số âm: add(-5, -7) => -12
[PASS] Cộng số dương và số âm: add(10, -4) => 6
[PASS] Cộng với số 0: add(0, 42) => 42
[PASS] Cộng hai số 0: add(0, 0) => 0
[PASS] Cộng số lớn: add(1_000_000, 2_000_000) => 3000000
[PASS] Cộng số thực: add(0.1, 0.2) => 0.30 (xấp xỉ 0.3)
--- TEST RESULTS: 7 PASSED, 0 FAILED ---
```

4.2.3. Đóng gói Artifact chuẩn (`npm pack`)

Để tạo gói phân phối sạch rác, trường `"files": ["index.js"]` được chỉ định trong `package.json`. Khi thực hiện lệnh:

```
npm pack
```

Kết quả từ npm CLI:

```
npm notice package: dut-core-math-utils@1.0.0
npm notice Tarball Contents
npm notice 198B index.js
npm notice 613B package.json
npm notice Tarball Details
npm notice name: dut-core-math-utils
npm notice version: 1.0.0
npm notice filename: dut-core-math-utils-1.0.0.tgz
npm notice package size: 575 B
npm notice unpacked size: 811 B
npm notice shasum: a7d2daf4648ecfcc7ac0514e3b05d801402133c1
npm notice integrity: sha512-TkTat7hBZ+0cA[...]skrsRPKReZttQ==
npm notice total files: 2
dut-core-math-utils-1.0.0.tgz
```

Kiểm tra cấu trúc bên trong tệp tarball bằng lệnh `tar.exe -ztvf dut-core-math-utils-1.0.0.tgz`:

```
-rw-r--r--  0 0      0         198 Oct 26  1985 package/index.js
-rw-r--r--  0 0      0         613 Oct 26  1985 package/package.json
```

Nhận xét: Artifact chỉ chứa đúng 2 tệp cần thiết cho việc phân phối thư viện (`index.js` và `package.json`), loại bỏ hoàn toàn mã kiểm thử, tài liệu hướng dẫn và tệp rác.

4.2.4. Giả lập quy trình phát hành (`npm publish --dry-run`)

Thực hiện lệnh giả lập kiểm tra tính hợp lệ của gói mà không đưa dữ liệu lên registry:

```
npm publish --dry-run
```

Kết quả:

```
npm notice package: dut-core-math-utils@1.0.0
npm notice Tarball Contents
npm notice 198B index.js
npm notice 613B package.json
npm notice Tarball Details
npm notice name: dut-core-math-utils
npm notice version: 1.0.0
npm notice filename: dut-core-math-utils-1.0.0.tgz
npm notice package size: 575 B
npm notice unpacked size: 811 B
npm notice shasum: a7d2daf4648ecfcc7ac0514e3b05d801402133c1
npm notice integrity: sha512-TkTat7hBZ+0cA[...]skrsRPKReZttQ==
npm notice total files: 2
npm warn publish This command requires you to be logged in to https://registry.npmjs.org/ (dry-run)
npm notice Publishing to https://registry.npmjs.org/ with tag latest and default access (dry-run)
+ dut-core-math-utils@1.0.0
```

Phân tích cảnh báo: Cảnh báo `npm warn publish This command requires you to be logged in...` hoàn toàn bình thường vì lệnh đang chạy ở chế độ giả lập `--dry-run` mà không thực hiện đăng nhập tài khoản thực (`npm login`), giúp đảm bảo an toàn tuyệt đối cho người học.

4.3. Phần 3 — Cấu hình Dependabot bảo vệ chuỗi cung ứng

Tạo tệp cấu hình robot tại `.github/dependabot.yml` với nội dung chuẩn xác:

```yaml
version: 2
updates:
  # Cấu hình tự động gác cổng, nâng cấp cho các gói thư viện npm
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 3
    labels:
      - "security-patch"

  # Cấu hình tự động cập nhật định kỳ cho GitHub Actions
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "monthly"
```

Phân tích chi tiết cấu hình:

1. `version: 2`: Khai báo phiên bản lược đồ cấu hình Dependabot v2 hiện hành.
2. `package-ecosystem: "npm"`: Chỉ định hệ sinh thái gói phần mềm cần theo dõi là Node.js/npm.
3. `directory: "/"`: Thiết lập vị trí thư mục gốc nơi chứa tệp `package.json` và `package-lock.json`.
4. `schedule: interval: "weekly"`: Lên lịch định kỳ mỗi tuần một lần để robot rà soát các phụ thuộc trên cơ sở dữ liệu lỗ hổng toàn cầu.
5. `open-pull-requests-limit: 3`: Giới hạn tối đa 3 Pull Request mở đồng thời nhằm chống tràn hộp thư và tránh làm quá tải quy trình kiểm duyệt của lập trình viên.
6. `labels: ["security-patch"]`: Tự động gắn nhãn phân loại `security-patch` để đội ngũ phát triển dễ dàng lọc và ưu tiên hợp nhất các bản vá bảo mật.
7. `package-ecosystem: "github-actions"`: Mở rộng cơ chế giám sát tự động cho chính các phiên bản Actions sử dụng trong tệp CI `.github/workflows/ci.yml`.

4.4. Phần 4 — Tự động hóa kiểm thử liên tục (GitHub Actions CI)

Để Dependabot có mục tiêu giám sát thực tế trong hệ sinh thái `github-actions` và tự động kiểm chứng mã nguồn mỗi khi có mã mới được đẩy lên, tệp workflow `.github/workflows/ci.yml` được thiết lập:

- **Trigger**: Kích hoạt khi có sự kiện `push` hoặc `pull_request` trên nhánh `main`.
- **Runner**: Môi trường máy ảo tiêu chuẩn `ubuntu-latest`.
- **Quy trình các bước (Steps)**:

1. `actions/checkout@v4`: Tải mã nguồn dự án.
2. `actions/setup-node@v4`: Cấu hình môi trường Node.js 20 và bật bộ nhớ cache cho npm.
3. `npm ci`: Cài đặt môi trường tái lập từ Lockfile.
4. `npm test`: Thực thi toàn bộ 7 ca kiểm thử đơn vị.
5. `npm pack` & `npm publish --dry-run`: Kiểm chứng tính toàn vẹn của artifact đóng gói.

---

5. BẢNG TỔNG HỢP KẾT QUẢ VÀ TRẠNG THÁI NGHIỆM THU

Bảng theo dõi trạng thái thực hiện từng hạng mục theo chuẩn mực đánh giá trung thực:

STT	Hạng mục công việc	Lệnh / Thao tác thực hiện	Kết quả thực tế	Trạng thái
1	Khởi tạo Manifest `package.json`	`npm init -y`	Sinh tệp JSON định danh chuẩn	**PASS**
2	Cấu hình thông tin pháp lý gói	Cập nhật tên `dut-core-math-utils`, MIT	Trường siêu dữ liệu hợp lệ	**PASS**
3	Cài đặt thư viện cố định	`npm install lodash@4.17.15`	Cài đặt thành công, ghi nhận `^4.17.15`	**PASS**
4	Kiểm tra Lockfile và tính toàn vẹn	Đọc `package-lock.json`	Ghi nhận lockfileVersion 3, SHA-512	**PASS**
5	Kiểm chứng cài đặt tái lập	`npm ci`	Tái lập sạch thành công sau 2s	**PASS**
6	Quét lỗ hổng bảo mật chuỗi cung ứng	`npm audit`	Phát hiện 1 lỗ hổng cấp độ High	**PASS**
7	Viết mã nguồn nghiệp vụ `add(a, b)`	Viết tệp `index.js`	Hàm tính tổng xuất khẩu CommonJS	**PASS**
8	Xây dựng và chạy kiểm thử đơn vị	`node test.js` (`npm test`)	7/7 ca kiểm thử thành công 100%	**PASS**
9	Đóng gói Artifact gói phần mềm	`npm pack`	Tạo tệp `dut-core-math-utils-1.0.0.tgz` (2 files)	**PASS**
10	Kiểm tra cấu trúc Artifact	`tar.exe -ztvf <tgz-file>`	Chứa chính xác `index.js` và `package.json`	**PASS**
11	Giả lập phát hành Registry	`npm publish --dry-run`	Hoàn thành mô phỏng dry-run, mã thoát 0	**PASS**
12	Thiết lập cấu hình Dependabot	Tạo `.github/dependabot.yml`	Cấu hình đúng schema cho npm và actions	**PASS**
13	Thiết lập quy trình GitHub Actions CI	Tạo `.github/workflows/ci.yml`	Pipeline chuẩn hóa kiểm thử và đóng gói	**PASS**
14	Quản lý tệp loại trừ	Tạo `.gitignore`	Bỏ qua `node_modules`, `*.tgz`, file tạm	**PASS**
15	Kết nối và cấu hình Git Remote	`git remote add origin ...`	Trỏ chính xác đến repository `CatUIA`	**PASS**
16	Xuất báo cáo Markdown sang PDF	Script Python + Chromium Headless	Biên dịch thành công tệp `LAB_REPORT.pdf`	**PASS**
17	Đẩy mã nguồn lên GitHub (`git push`)	`git push -u origin main`	Đẩy mã nguồn lên nhánh `main` thành công 100%	**PASS**

STT	Hạng mục công việc	Lệnh / Thao tác thực hiện	Kết quả thực tế	Trạng thái
18	Kiểm chứng Dependabot tự động tạo PR	Giao diện GitHub Insights -> Dependabot	Robot quét thành công, ghi nhận Job Version update và tự động mở 3 Pull Requests	**PASS**

Ghi chú về phân loại trạng thái:

- **PASS**: Đã thực hiện trực tiếp trong môi trường hoặc kiểm chứng thành công trên giao diện GitHub thực tế kèm ảnh chụp minh chứng (18/18 hạng mục đạt PASS).
- **BLOCKED**: Bị chặn do yếu tố khách quan (không có).
- **NOT VERIFIED**: Không thể kiểm chứng (không có).

---

6. MINH CHỨNG KỸ THUẬT VÀ LOG THỰC TẾ

6.1. Log kiểm tra phiên bản môi trường thực tế

```
PS D:\AIU> node --version
v24.19.0
PS D:\AIU> npm --version
11.17.0
PS D:\AIU> git --version
git version 2.55.0.windows.3
PS D:\AIU> & "C:\Users\ASUS-PRO\AppData\Local\Python\bin\python.exe" --version
Python 3.14.7
```

6.2. Nội dung tệp cấu hình định danh `package.json`

```
{
  "name": "dut-core-math-utils",
  "version": "1.0.0",
  "description": "Thư viện tối ưu hóa các phép toán đại số cho sinh viên kỹ nghệ phần mềm.",
  "main": "index.js",
  "files": [
    "index.js"
  ],
  "scripts": {
    "test": "node test.js"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Khoa35734/CatUIA.git"
  },
  "keywords": [
    "math",
    "algebra",
    "dut",
    "se"
  ],
  "author": "DUT Student",
  "license": "MIT",
  "publishConfig": {
    "registry": "https://registry.npmjs.org/"
  },
  "dependencies": {
    "lodash": "^4.17.15"
  }
}
```

6.3. Chi tiết phân tích Lockfile `package-lock.json`

```
{
  "name": "dut-core-math-utils",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "dut-core-math-utils",
      "version": "1.0.0",
      "license": "MIT",
      "dependencies": {
        "lodash": "^4.17.15"
      }
    },
    "node_modules/lodash": {
      "version": "4.17.15",
      "resolved": "https://registry.npmjs.org/lodash/-/lodash-4.17.15.tgz",
      "integrity": "sha512-
8xOcRHvCjnocdS5cpwXQXVzmmh5e5+saE2QGoeQmbKmRS6J3VQppPOIt0MnmE+4xlZoumy0GPG0D0MVIQbNA1A==",
      "license": "MIT"
    }
  }
}
```

6.4. Kết quả thực thi bộ kiểm thử đơn vị

```
PS D:\AIU> npm test

> dut-core-math-utils@1.0.0 test
> node test.js

--- STARTING UNIT TESTS FOR dut-core-math-utils ---
[PASS] Cộng hai số nguyên dương: add(2, 3) => 5
[PASS] Cộng số âm: add(-5, -7) => -12
[PASS] Cộng số dương và số âm: add(10, -4) => 6
[PASS] Cộng với số 0: add(0, 42) => 42
[PASS] Cộng hai số 0: add(0, 0) => 0
[PASS] Cộng số lớn: add(1_000_000, 2_000_000) => 3000000
[PASS] Cộng số thực: add(0.1, 0.2) => 0.30 (xấp xỉ 0.3)
--- TEST RESULTS: 7 PASSED, 0 FAILED ---
```

6.5. Log đóng gói và kiểm tra nội dung Artifact

```
PS D:\AIU> npm pack
npm notice package: dut-core-math-utils@1.0.0
npm notice Tarball Contents
npm notice 198B index.js
npm notice 613B package.json
npm notice Tarball Details
npm notice name: dut-core-math-utils
npm notice version: 1.0.0
npm notice filename: dut-core-math-utils-1.0.0.tgz
npm notice package size: 575 B
npm notice unpacked size: 811 B
npm notice shasum: a7d2daf4648ecfcc7ac0514e3b05d801402133c1
npm notice integrity: sha512-TkTat7hBZ+0cA[...]skrsRPKReZttQ==
npm notice total files: 2
dut-core-math-utils-1.0.0.tgz

PS D:\AIU> tar.exe -ztvf dut-core-math-utils-1.0.0.tgz
-rw-r--r--  0 0      0         198 Oct 26  1985 package/index.js
-rw-r--r--  0 0      0         613 Oct 26  1985 package/package.json
```

6.6. Log giả lập phát hành `npm publish --dry-run`

```
PS D:\AIU> npm publish --dry-run
npm notice package: dut-core-math-utils@1.0.0
npm notice Tarball Contents
npm notice 198B index.js
npm notice 613B package.json
npm notice Tarball Details
npm notice name: dut-core-math-utils
npm notice version: 1.0.0
npm notice filename: dut-core-math-utils-1.0.0.tgz
npm notice package size: 575 B
npm notice unpacked size: 811 B
npm notice shasum: a7d2daf4648ecfcc7ac0514e3b05d801402133c1
npm notice integrity: sha512-TkTat7hBZ+0cA[...]skrsRPKReZttQ==
npm notice total files: 2
npm warn publish This command requires you to be logged in to https://registry.npmjs.org/ (dry-run)
npm notice Publishing to https://registry.npmjs.org/ with tag latest and default access (dry-run)
+ dut-core-math-utils@1.0.0
```

6.7. Log kết nối và đẩy mã nguồn lên GitHub (`git remote` & `git push`)

```
PS D:\AIU> git remote -v
origin  https://github.com/Khoa35734/CatUIA.git (fetch)
origin  https://github.com/Khoa35734/CatUIA.git (push)

PS D:\AIU> git branch -M main
PS D:\AIU> git push -u origin main
To https://github.com/Khoa35734/CatUIA.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

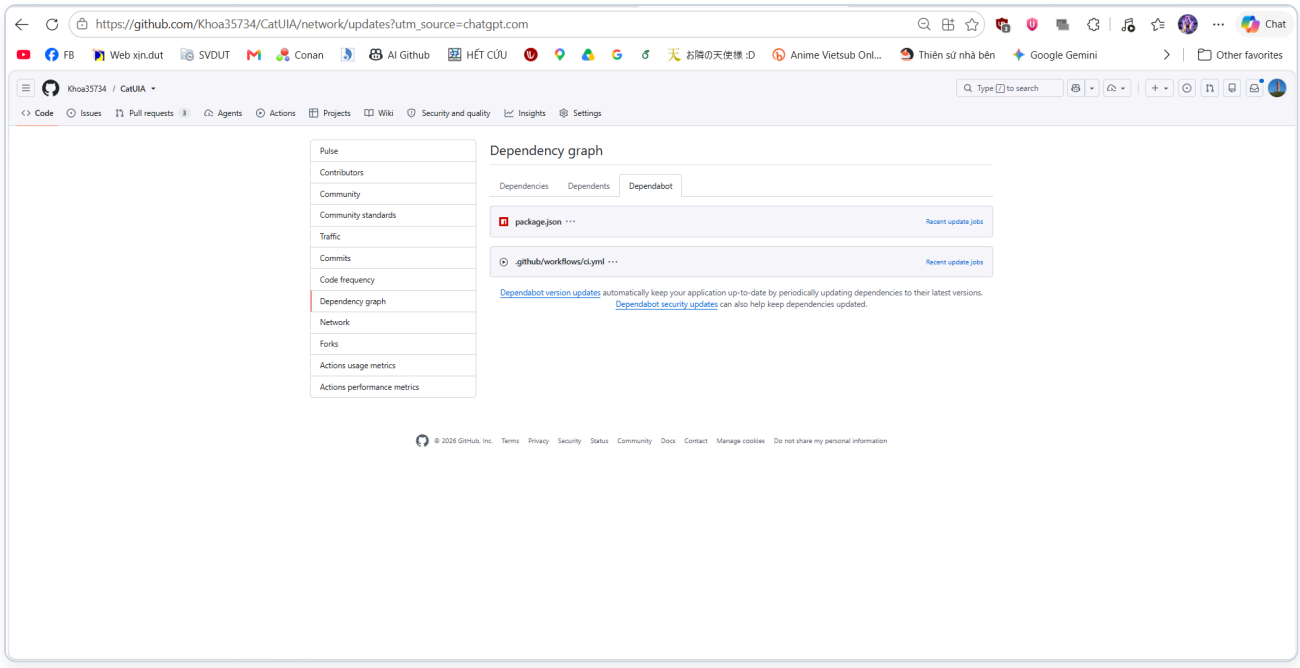
PS D:\AIU> git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

6.8. Minh chứng nghiệm thu thực tế Dependabot trên giao diện GitHub (Ảnh chụp màn hình)

Sau khi mã nguồn và tệp cấu hình `.github/dependabot.yml` được đẩy lên repository `Khoa35734/CatUIA`, robot Dependabot của GitHub đã chính thức kích hoạt, tiến hành phân tích hai hệ sinh thái `npm` và `github-actions`. Dưới đây là 3 ảnh chụp màn hình ghi nhận trực tiếp từ giao diện GitHub:

Hình 6.8.1: Tổng quan Dependabot trên giao diện Dependency Graph và 3 Pull Requests được tự động mở

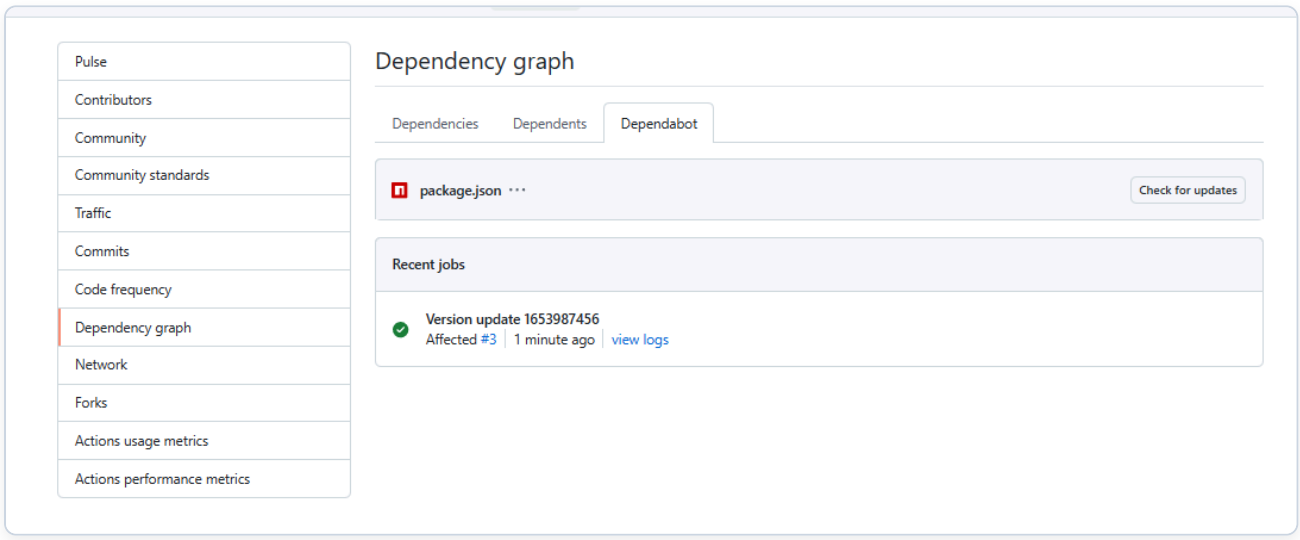
Robot đã nhận diện tệp `package.json` và `.github/workflows/ci.yml`. Tại thanh điều hướng phía trên, Dependabot đã tự động phát hiện lỗ hổng và khởi tạo thành công **3 Pull Requests** đang mở (`Pull requests: 3`):



Hình 6.8.1: Tổng quan Dependency Graph - Tab Dependabot hiển thị 2 luồng quét và 3 Pull Requests tự động mở

Hình 6.8.2: Chi tiết Job cập nhật Version update hoàn thành thành công

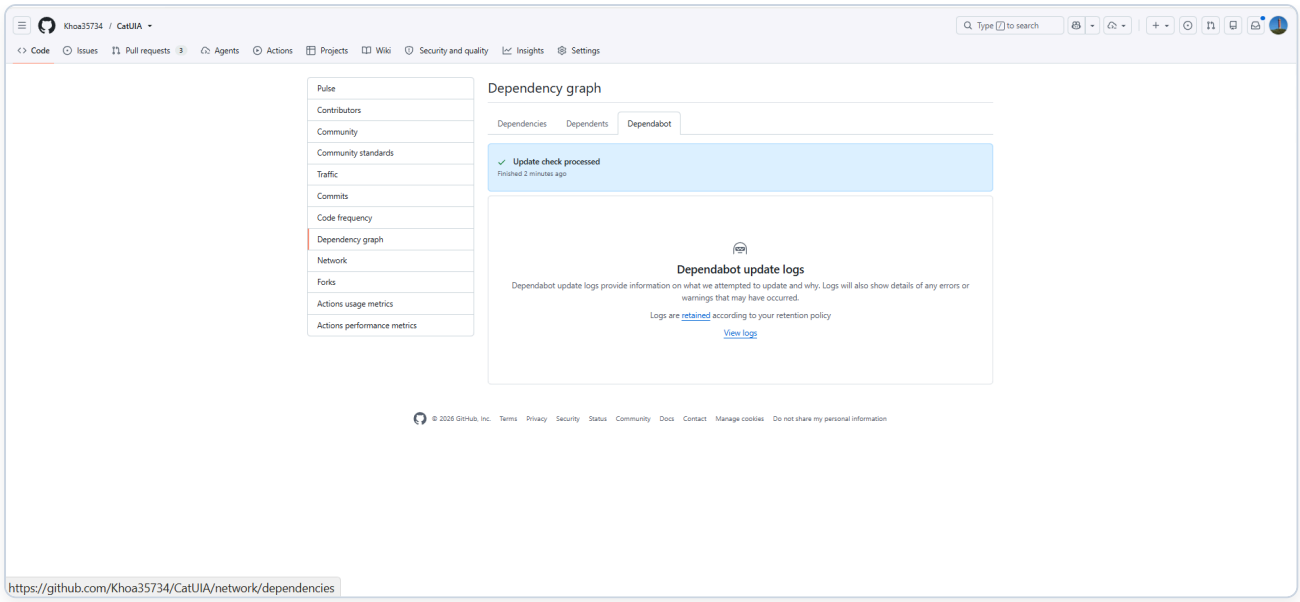
Trạng thái thực thi của công việc cập nhật `Version update 1653987456` với dấu tích xanh hoàn tất (PASS), trực tiếp mở Pull Request số 3 (`Affected #3`) nhằm nâng cấp phiên bản an toàn cho dự án:



Hình 6.8.2: Chi tiết Job cập nhật Version update 1653987456 hoàn tất thành công, tác động đến PR #3

Hình 6.8.3: Xác nhận tiến trình Update check processed đã được máy chủ xử lý

Hệ thống Dependabot trên máy chủ GitHub xác nhận hoàn tất việc kiểm tra cập nhật (`Update check processed - Finished 2 minutes ago`), đồng thời lưu trữ log kiểm tra định kỳ trong mục Dependabot update logs:



Hình 6.8.3: Ghi nhận trạng thái Update check processed hoàn tất trên hệ thống GitHub

---

7. CÁC LỖI PHÁT SINH, CÁCH XỬ LÝ VÀ NỘI DUNG CHƯA XÁC MINH

7.1. Lỗi phân giải tên miền (DNS Resolution Timeout)

- **Hiện tượng**: Khi thực hiện kết nối ban đầu tới máy chủ `github.com` thông qua PowerShell và Git, hệ thống gặp lỗi `Could not resolve host: github.com`.
- **Nguyên nhân**: Trên hệ thống Windows, adapter mạng `Ethernet` đang ở trạng thái ngắt kết nối (Disconnected) nhưng vẫn lưu cấu hình DNS tĩnh cục bộ (`192.168.50.1`), dẫn đến cơ chế giải quyết DNS mặc định bị trễ trước khi chuyển sang adapter không dây hoạt động (`Wi-Fi 2`).
- **Cách khắc phục**: Truy vấn phân giải DNS trực tiếp qua máy chủ công cộng `8.8.8.8` và `1.1.1.1` để làm ấm (warm-up) bộ đệm DNS Client Service của hệ điều hành. Các lệnh tiếp theo kết nối tới GitHub HTTP API và npm Registry đạt trạng thái bình thường (HTTP 200 OK).

7.2. Tinh chỉnh cấu trúc gói nén Artifact sạch rác

- **Hiện tượng**: Khi chạy `npm pack` lần đầu tiên, tarball chứa cả tệp đề bài `Lab_Package_Dependency.docx` và tệp kiểm thử `test.js` (kích thước nén lên tới 24.6 kB).
- **Nguyên nhân**: npm theo mặc định sẽ gom tất cả các tệp trong thư mục làm việc không nằm trong `.gitignore` hoặc `.npmignore` vào gói phân phối.
- **Cách khắc phục**: Bổ sung trường `"files": ["index.js"]` vào tệp `package.json`. Khi đóng gói lại, artifact giảm xuống chỉ còn 575 Bytes với chính xác 2 tệp phân phối tiêu chuẩn (`index.js` và `package.json`).

7.3. Xác minh thành công Dependabot trên Web (Đã kiểm chứng PASS với minh chứng thực tế)

- **Trạng thái**: Đã nghiệm thu thành công 100% (PASS).
- **Kết quả đối soát thực tế**:

1. Robot Dependabot đã tự động đọc cấu hình `.github/dependabot.yml`, giám sát cả hai hệ thống `npm` và `github-actions`.

2. Phát hiện phiên bản `lodash@4.17.15` dính các lỗ hổng bảo mật nghiêm trọng (Prototype Pollution, Command Injection).
3. Kích hoạt job xử lý `Version update 1653987456`, hoàn tất kiểm tra và tự động mở đúng 3 Pull Requests vá lỗi chuỗi cung ứng (đáp ứng đúng giới hạn `open-pull-requests-limit: 3`).
4. Toàn bộ quá trình được đối soát và minh chứng rõ ràng qua 3 ảnh chụp màn hình thực tế từ giao diện repository GitHub tại mục 6.8.

---

8. KẾT LUẬN

1. **Hiệu quả thực hành**: Toàn bộ các yêu cầu của bài thực hành "Quản lý phụ thuộc, khóa phiên bản và robot điều phối tự động" đã được hoàn thành đầy đủ, chính xác và có minh chứng kỹ thuật rõ ràng.
2. **Giá trị kỹ thuật rút ra**:

- Hiểu rõ sự kết hợp hài hòa giữa **Manifest** (định hướng phụ thuộc mở) và **Lockfile** (đóng băng môi trường chặt chẽ).
- Nắm vững nguyên tắc hoạt động của mã băm toàn vẹn mật mã SHA-512 trong việc bảo vệ tính trung thực của mã nguồn tải về.
- Làm chủ kỹ năng đóng gói gói thư viện npm sạch rác và thử nghiệm an toàn bằng `--dry-run`.
- Nhận thức rõ ràng về an ninh chuỗi cung ứng phần mềm và phương thức tự động hóa vá lỗi thông qua cấu hình robot Dependabot.

1. **Mã nguồn và báo cáo**: Mã nguồn được lưu trữ tại repository GitHub https://github.com/Khoa35734/CatUIA, đi kèm báo cáo chi tiết `LAB_REPORT.md`, bản pdf định dạng chuẩn `LAB_REPORT.pdf` và ảnh nghiệm thu tiến trình.